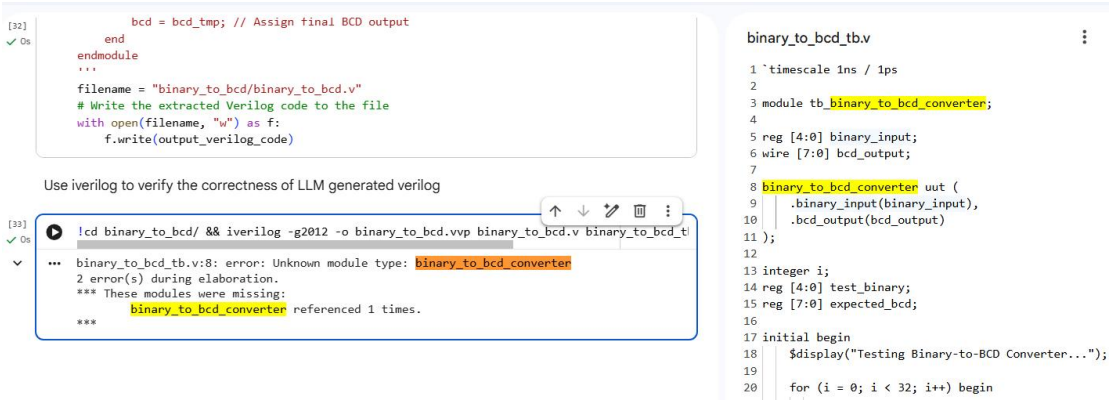
ChipChat Tutorial Assignment

Jc13605 Jiangyue Chu

Example A: binary_to_bcd (combinational converter)

Iteration 1:



In the first iteration, I wrote the prompt to generate the module name is binary_to_bcd.v. I didn't notice that in testbench, the required module name is binary_to_bcd_converter, so there is a mismatch on module name.

Iteration 2:

Error:

Prompt:

```
Write synthesizable SystemVerilog (or Verilog-2001) for a purely combinational binary-to-BCD converter.


Module name: binary_to_bcd_converter

Ports:

- input  logic [4:0] binary

- output logic [7:0] bcd   // bcd[7:4] = tens digit, bcd[3:0] = ones digit


Function:

- Convert binary (0..31) into BCD.

- Example: 0 -> 8'h00, 9 -> 8'h09, 10 -> 8'h10, 31 -> 8'h31.
```

Code:

```
module binary_to_bcd_converter(

    input [4:0] binary_input,

    output [7:0] bcd_output

);
```

```
    reg [3:0] tens;
    reg [3:0] ones;
    reg [4:0] temp;
    integer i;

    always @(*) begin
        tens = 4'b0000;
        ones = 4'b0000;
        temp = binary_input;

        for (i = 0; i < 5; i = i + 1) begin
            if (ones >= 4'b0100)
                ones = ones + 4'b0011;

            if (tens >= 4'b0100)
                tens = tens + 4'b0011;

            {tens, ones} = {tens, ones} << 1;
            ones[0] = temp[4];
            temp = {temp[3:0], 1'b0};
        end
    end

    assign bcd_output = {tens, ones};

endmodule
```

The initial LLM-generated design failed several test cases due to an incorrect implementation of the double-dabble algorithm.
The main issues were an incorrect add-3 threshold and an incorrect ordering of the add-3 and shift operations, which caused wrong BCD outputs for some inputs (e.g., input 5 producing output 8).
The design was corrected by reimplementing the standard double-dabble algorithm using a single 8-bit temporary register and applying the add-3 step before each shift.
After this fix, the design passed all test cases in the provided testbench.

Iteration 3:

Final Prompt:

```
Write Verilog-2001 for a combinational 5-bit binary to 8-bit BCD converter using double-dabble.

Module name: binary_to_bcd_converter
Ports: input [4:0] binary_input, output [7:0] bcd_output (bcd_output[7:4]=tens, [3:0]=ones)
```

```
Must use double-dabble with correct order each iteration:
1) if any BCD digit >= 5 add 3
2) shift left and shift in next binary bit from MSB to LSB

Output ONLY the module code.
```

Final Code:

```
module binary_to_bcd_converter(
    input [4:0] binary_input,
    output reg [7:0] bcd_output
);

    integer i;

    always @* begin
        // Initialize BCD output
        bcd_output = 8'b00000000;

        // Perform double-dabble conversion
        for (i = 4; i >= 0; i = i - 1) begin
            // Step 1: If any BCD digit >= 5, add 3
            if (bcd_output[7:4] >= 5)
                bcd_output[7:4] = bcd_output[7:4] + 3;
            if (bcd_output[3:0] >= 5)
                bcd_output[3:0] = bcd_output[3:0] + 3;

            // Step 2: Shift left and shift in the next binary bit
            bcd_output = {bcd_output[6:0], binary_input[i]};
        end
    end

endmodule
```

```
!cd binary_to_bcd/ && iverilog -g2012 -o binary_to_bcd.vvp binary_to_bcd.v binary_to_bcd_tb.v && vvp binary_to_bcd.vvp
```

```
Testing Binary-to-BCD Converter...
VCD info: dumpfile my_design.vcd opened for output.
All test cases passed!
```

Example B: sequence_detector (FSM sequence matching)

Prompt：

```
I need help writing a Verilog module for a simple traffic light controller.

The design should be a synchronous finite state machine driven by a clock. It has an active-low reset.
```

Inputs:

- clk

- reset (active low)

Outputs:

- red

- yellow

- green

The behavior should be as follows:

- After reset, the traffic light starts in the red state.

- The light stays red for 32 clock cycles, then switches to green.

- The light stays green for 20 clock cycles, then switches to yellow.

- The light stays yellow for 10 clock cycles, then switches back to red.

- This sequence should repeat continuously.

Only one light should be asserted at any time.

A simple FSM with a state register and a counter is fine. Please make sure the reset behavior and timing are handled correctly.

The code should be synthesizable Verilog (no delays, no initial blocks).

Please output only the complete module code.

Code：

```
module traffic_light_controller (
    input wire clk,
    input wire reset_n,
    output reg red,
    output reg yellow,
    output reg green
);

    // State encoding
    typedef enum reg [1:0] {
        RED = 2'b00,
        GREEN = 2'b01,
        YELLOW = 2'b10
    } state_t;

    state_t current_state, next_state;
    reg [5:0] counter; // 6 bits can count up to 63

    // State register
```

```
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        current_state <= RED;
        counter <= 0;
    end else begin
        // State transition
        if (counter == get_state_duration(current_state)) begin
            current_state <= next_state;
            counter <= 0;
        end else begin
            counter <= counter + 1;
        end
    end
end

// Next state logic
always @(*) begin
    case (current_state)
        RED: next_state = GREEN;
        GREEN: next_state = YELLOW;
        YELLOW: next_state = RED;
        default: next_state = RED; // Fallback
    endcase
end

// Output logic
always @(*) begin
    case (current_state)
        RED: begin
            red = 1;
            yellow = 0;
            green = 0;
        end
        GREEN: begin
            red = 0;
            yellow = 0;
            green = 1;
        end
        YELLOW: begin
            red = 0;
            yellow = 1;
            green = 0;
        end
        default: begin
```

```
                red = 0;
                yellow = 0;
                green = 0; // Default safe state
            end
        endcase
    end

    // Function to return the duration for each state
    function [5:0] get_state_duration(input state_t state);
        begin
            case (state)
                RED: get_state_duration = 32; // 32 clock cycles for red
                GREEN: get_state_duration = 20; // 20 clock cycles for green
                YELLOW: get_state_duration = 10; // 10 clock cycles for yellow
                default: get_state_duration = 0; // Default case
            endcase
        end
    endfunction

endmodule
```

```
!cd traffic_light/ && iverilog -g2012 -o traffic_light.vvp traffic_light.v traffic_light_tb.v && vvp traffic_light.vvp
```

```
Test case 1 passed - FSM transitioned to    RED after 0 clock cycles.
Test case 2 passed - FSM transitioned to  GREEN after 32 clock cycles.
Test case 3 passed - FSM transitioned to YELLOW after 20 clock cycles.
Test case 4 passed - FSM transitioned to    RED after 7 clock cycles.
All test cases passed!
```

Part III:

For Part III, I chose a spec change based on the traffic light FSM example. The original design only outputs one-hot signals (red, yellow, green). I extended the specification by adding a 2-bit output state_code to explicitly indicate the current FSM state.

Design Changes

The FSM behavior remains unchanged: RED for 32 cycles, GREEN for 20 cycles, and YELLOW for 7 cycles, with transitions controlled by a counter when enable=1. The new state_code output encodes the state as 00=RED, 01=GREEN, and 10=YELLOW, and is always consistent with the one-hot light outputs.

Verification

I modified the original testbench to include checks for the new state_code signal while keeping the original functional tests. The extended design was compiled and simulated using iverilog and passed all test cases.

Prompt:

```
I already have a working traffic light FSM, and I would like to extend it slightly.
```

Please generate a synthesizable Verilog-2001 module named traffic_light_fsm with the following interface:

- input wire clk
- input wire reset_n   (active-low reset)
- input wire enable
- output reg red
- output reg yellow
- output reg green
- output reg [1:0] state_code

The behavior of the traffic light should remain the same as before:
- After reset, the FSM starts in the RED state.
- When enable = 1:
  - Stay in RED for 32 clock cycles, then transition to GREEN.
  - Stay in GREEN for 20 clock cycles, then transition to YELLOW.
  - Stay in YELLOW for 7 clock cycles, then transition back to RED.
- When enable = 0, the FSM should hold its current state and counter.
- Only one of red, yellow, or green should be asserted at any time.

Extension requirement:
- Add the 2-bit output state_code to explicitly encode the current FSM state:
  - 2'b00 = RED
  - 2'b01 = GREEN
  - 2'b10 = YELLOW
- The value of state_code should always be consistent with the red/yellow/green outputs.

Implementation notes:
- A simple FSM with a state register and a counter is sufficient.
- Use plain Verilog-2001 style (no typedef, enum, logic, always_comb, or SystemVerilog-only syntax).
- Use a synchronous always block for state and counter updates, and a combinational block for outputs.
- No delays, no initial blocks.

Please output only the complete module code.

Code:

```
module traffic_light_fsm (
    input wire clk,
    input wire reset_n,
    input wire enable,
    output reg red,
    output reg yellow,
    output reg green,
    output reg [1:0] state_code
);
```

```
// State encoding
localparam RED    = 2'b00;
localparam GREEN  = 2'b01;
localparam YELLOW = 2'b10;

// Timer values
localparam RED_TIME    = 32;
localparam GREEN_TIME  = 20;
localparam YELLOW_TIME = 7;

reg [1:0] state;
reg [4:0] counter; // Enough for counting up to 32

// State and counter updates
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        state <= RED;
        counter <= 0;
    end else if (enable) begin
        case (state)
            RED: begin
                if (counter == RED_TIME - 1) begin
                    state <= GREEN;
                    counter <= 0;
                end else
                    counter <= counter + 1;
            end

            GREEN: begin
                if (counter == GREEN_TIME - 1) begin
                    state <= YELLOW;
                    counter <= 0;
                end else
                    counter <= counter + 1;
            end

            YELLOW: begin
                if (counter == YELLOW_TIME - 1) begin
                    state <= RED;
                    counter <= 0;
                end else
                    counter <= counter + 1;
            end
```

```
            default: begin
                state <= RED;
                counter <= 0;
            end
        endcase
    end
end

// Output logic
always @* begin
    // Default outputs
    red = 0;
    yellow = 0;
    green = 0;
    state_code = state;

    case (state)
        RED:    red = 1;
        GREEN:  green = 1;
        YELLOW: yellow = 1;
    endcase
end

endmodule
```

```
!cd traffic_light/ && iverilog -g2012 -o traffic_light.vvp traffic_light.v traffic_light_tb.v && vvp traffic_light.vvp
```

```
All test cases passed!
```